

Unidad: 3 Actividad entregable: Simulación de pruebas de seguridad en fases del desarrollo de software seguro.

Diana Zoraida Romero Ramírez

ID: 949490.

Corporación Universitaria Minuto de Dios UNIMINUTO

Programa: Ingeniería de Sistemas

NRC: 40-63386 Desarrollo de software seguro.

Docente: Edwin Albeiro Ramos Villamil.

Mayo 02, 2025

Bogotá D.C

1. Introducción

En el desarrollo de software, las pruebas cumplen un papel fundamental para garantizar la calidad, el correcto funcionamiento y la confiabilidad de los sistemas. A través de diferentes tipos de pruebas, es posible identificar errores, validar requerimientos y asegurar que la aplicación cumpla con las expectativas del usuario.

En el presente caso de estudio se aplicará un proceso de pruebas sobre la aplicación web Software Seguro, la cual corresponde a un sistema tipo comercio electrónico desarrollado en PHP con conexión a base de datos MySQL. Esta aplicación permite la gestión de usuarios, roles y autenticación mediante inicio de sesión, siendo estos módulos críticos para su funcionamiento.

Para el desarrollo de este caso, se implementarán pruebas unitarias y pruebas de integración utilizando la herramienta PHPUnit, con el fin de validar tanto escenarios correctos (pruebas positivas) como escenarios incorrectos (pruebas negativas). Además, se busca identificar posibles fallos en el sistema, especialmente en el manejo de errores y validaciones.

Finalmente, los resultados obtenidos permitirán evaluar el comportamiento del sistema, evidenciar su funcionamiento mediante pruebas reales y proponer mejoras en aquellos puntos donde se detecten debilidades.

2. Propósito

El propósito de este caso de estudio es definir y ejecutar un proceso de pruebas para la aplicación web Software Seguro, con el fin de validar el correcto funcionamiento de sus principales módulos, especialmente aquellos relacionados con la autenticación de usuarios, la gestión de roles y la consulta de información almacenada en la base de datos.

Para el desarrollo de este proceso se toma como referencia la estructura de pruebas planteada en el documento base, donde se establece la importancia de realizar pruebas en diferentes etapas del desarrollo con el objetivo de asegurar la calidad del software.

En este sentido, se aplicarán pruebas unitarias y pruebas de integración utilizando la herramienta PHPUnit, permitiendo evaluar tanto escenarios correctos (pruebas positivas) como escenarios incorrectos (pruebas negativas). Esto facilitará la identificación de errores, la validación de requerimientos y la generación de evidencia del comportamiento del sistema.

Finalmente, este proceso busca no solo comprobar que el sistema funcione correctamente, sino también detectar posibles debilidades en el manejo de errores y en la validación de datos, aportando así a la mejora continua del software.

3. Contexto actual

La aplicación web Software Seguro es un sistema desarrollado en PHP con arquitectura basada en modelos, vistas y controladores (MVC), que funciona en un entorno local mediante XAMPP y utiliza una base de datos MySQL para el almacenamiento de la información.

El sistema presenta una interfaz tipo comercio electrónico, donde se visualiza una página principal con diseño moderno, navegación superior y diferentes secciones como inicio, productos, contacto e inicio de sesión. Sin embargo, más allá de la interfaz visual, el sistema cuenta con módulos funcionales orientados a la gestión interna, como lo son el manejo de usuarios y roles.

Actualmente, la aplicación permite realizar procesos como el inicio de sesión mediante correo electrónico y contraseña, la consulta de usuarios registrados, la

administración de roles y la relación entre estos elementos dentro de la base de datos. Estos procesos son fundamentales para el control de acceso y la seguridad del sistema.

No obstante, durante el desarrollo del sistema no se implementó inicialmente un proceso formal de pruebas, lo cual puede generar errores en tiempo de ejecución, especialmente en escenarios donde se ingresan datos incorrectos o inexistentes. Por esta razón, se hace necesario aplicar un proceso de pruebas que permita validar el comportamiento del sistema y detectar posibles fallos.

4. Objetivo

El objetivo de este caso de estudio es aplicar un proceso de pruebas a la aplicación web Software Seguro, con el fin de evaluar el correcto funcionamiento de sus principales funcionalidades, especialmente aquellas relacionadas con el inicio de sesión, la gestión de usuarios y la administración de roles.

De manera específica, se busca implementar pruebas unitarias y pruebas de integración utilizando la herramienta PHPUnit, permitiendo validar tanto escenarios correctos como incorrectos. Esto incluye la verificación de datos válidos (pruebas positivas) y la detección de fallos ante datos inválidos o inexistentes (pruebas negativas).

Asimismo, se pretende identificar posibles errores en el sistema, como fallos en consultas a la base de datos o deficiencias en el manejo de excepciones, con el propósito de evidenciar debilidades y proponer mejoras que contribuyan a la calidad del software.

Finalmente, se espera obtener resultados verificables mediante la ejecución de pruebas automatizadas, los cuales servirán como evidencia del comportamiento del sistema y del cumplimiento de los requerimientos definidos.

5. Estrategia de pruebas

La estrategia de pruebas implementada para la aplicación web Software Seguro se basa en la validación de las funcionalidades más críticas del sistema, priorizando aquellas relacionadas con la autenticación de usuarios, la gestión de roles y la consulta de información desde la base de datos.

Para este caso de estudio, se decidió aplicar pruebas dinámicas, ya que permiten ejecutar directamente el sistema y observar su comportamiento en tiempo real. Estas pruebas se desarrollaron mediante el uso de la herramienta PHPUnit, lo cual permitió automatizar la validación de los diferentes escenarios definidos.

La estrategia incluye la aplicación de dos tipos principales de pruebas:

- **Pruebas unitarias:** Son enfocadas en validar el correcto funcionamiento de métodos específicos, como el proceso de inicio de sesión.
- **Pruebas de integración:** Son orientadas a verificar la interacción entre diferentes componentes del sistema, como la relación entre usuarios y roles en la base de datos.

Adicionalmente, se implementaron tanto pruebas positivas como negativas:

- **Las pruebas positivas:** Son las que validan que el sistema funcione correctamente cuando se ingresan datos válidos.
- **Las pruebas negativas:** Son las que permiten identificar fallos cuando se ingresan datos incorrectos, inexistentes o incompletos.

Durante la ejecución de las pruebas se detectaron algunos comportamientos inesperados, especialmente en el manejo de errores cuando no existen registros en la base de datos, lo cual evidencia la necesidad de mejorar la validación de datos y el control de excepciones en el sistema.

6. Alcance

El alcance del proceso de pruebas realizado en la aplicación web Software Seguro se centra en la validación de las funcionalidades implementadas en el sistema a nivel de lógica y acceso a datos, específicamente aquellas relacionadas con la autenticación de usuarios, la gestión de roles y la consulta de información desde la base de datos.

Las pruebas realizadas corresponden a pruebas unitarias y pruebas de integración, ejecutadas mediante la herramienta PHPUnit, utilizando datos reales almacenados en la base de datos del sistema.

- **Pruebas unitarias:** Se enfocaron en validar el funcionamiento del método de inicio de sesión, comprobando tanto escenarios correctos (credenciales válidas) como escenarios incorrectos (credenciales inválidas o usuarios inexistentes).
- **Pruebas de integración:** Se aplicaron para verificar la interacción entre los módulos del sistema, especialmente en consultas que involucran la relación entre usuarios y roles, así como la obtención de listas de datos desde la base de datos.

Las funcionalidades que fueron objeto de prueba son:

- Inicio de sesión de usuarios.
- Validación de credenciales incorrectas.
- Consulta de usuarios registrados.
- Consulta de roles disponibles.
- Obtención de roles por código.
- Verificación de la relación entre usuarios y roles mediante consultas integradas.

Asimismo, se incluyeron pruebas positivas y negativas, permitiendo evaluar tanto el comportamiento esperado del sistema como la respuesta ante datos incorrectos o inexistentes.

Es importante aclarar que este proceso de pruebas no incluye validaciones de interfaz gráfica, rendimiento, seguridad avanzada ni pruebas estructurales del código, ya que el enfoque del trabajo se centró únicamente en la validación funcional del sistema a través de pruebas automatizadas.

7. Criterios de suspensión y reanudación de pruebas

La ejecución de pruebas en la aplicación web Software Seguro podrá ser suspendida en los siguientes casos:

- No se cuenta con acceso a la base de datos o la conexión presenta fallos.
- Existen errores críticos que impiden la ejecución de las pruebas, como fallos en la configuración del entorno (por ejemplo, problemas con PHPUnit o el servidor local).
- El sistema presenta errores que generan interrupciones en la ejecución, como excepciones no controladas o fallos en consultas a la base de datos.

- No se dispone de datos válidos o consistentes para realizar las pruebas, como usuarios inexistentes o información incompleta.

Las pruebas podrán ser reanudadas una vez se hayan solucionado los problemas mencionados, asegurando que el entorno de ejecución esté correctamente configurado, la base de datos esté disponible y los datos necesarios para las pruebas se encuentren correctamente definidos.

8. Requerimientos de prueba

ID	Requerimiento de prueba	Tipo	Complejidad
1	Validar inicio de sesión con credenciales correctas	Funcionalidad	Alto
2	Validar inicio de sesión con credenciales incorrectas	Funcionalidad	Alto
3	Validar inicio de sesión con usuario inexistente	Funcionalidad	Alto
4	Consultar usuarios registrados en el sistema	Funcionalidad	Medio
5	Consultar roles disponibles	Funcionalidad	Medio
6	Obtener información de un rol por código	Funcionalidad	Medio
7	Validar relación entre usuarios y roles (consulta integrada)	Funcionalidad	Alto
8	Validar que la lista de usuarios no esté vacía	Funcionalidad	Simple
9	Validar que los datos retornados sean objetos tipo User	Funcionalidad	Simple
10	Validar comportamiento del sistema ante datos inválidos	Funcionalidad	Medio
11	Validar consultas directas a la base de datos	Funcionalidad	Medio
12	Validar manejo de errores en consultas sin resultados	Funcionalidad	Alto

9. Diseño de casos de prueba

Para el diseño de los casos de prueba de la aplicación web Software Seguro, se tomaron como base las funcionalidades que realmente fueron evaluadas durante la ejecución con PHPUnit. Los casos se enfocaron en validar el inicio de sesión, la consulta de usuarios, la consulta de roles y la relación entre usuarios y roles dentro de la base de datos.

Las pruebas fueron diseñadas considerando escenarios positivos y negativos. Los escenarios positivos permiten comprobar que el sistema responde correctamente cuando se ingresan datos válidos, mientras que los escenarios negativos permiten verificar el comportamiento del sistema cuando se utilizan datos incorrectos o inexistentes.

9.1 Pruebas funcionales automatizadas

Las pruebas realizadas corresponden a pruebas funcionales automatizadas, debido a que verifican el comportamiento del sistema frente a entradas y salidas esperadas. Para su ejecución se utilizó PHPUnit, herramienta que permitió comprobar si los métodos del sistema retornaban los resultados esperados.

En este caso, las pruebas se enfocaron en los siguientes componentes:

- Modelo User.php
- Clase de conexión DataBase.php
- Tabla USERS
- Tabla ROLES
- Relación entre usuarios y roles

9.2 Pruebas unitarias

Las pruebas unitarias se enfocaron en validar métodos específicos del modelo User, especialmente el método de inicio de sesión.

ID	Caso de prueba	Entrada	Resultado esperado	Tipo
CP01	Login con credenciales correctas	Correo y contraseña válidos	El sistema retorna un usuario válido	Positiva
CP02	Login con credenciales incorrectas	Correo falso y contraseña incorrecta	El sistema retorna false	Negativa
CP03	Login con usuario inexistente	Usuario no registrado	El sistema retorna false	Negativa

9.3 Pruebas de integración

Las pruebas de integración permitieron validar la comunicación entre el modelo, la conexión a base de datos y las tablas relacionadas.

ID	Caso de prueba	Entrada	Resultado esperado	Tipo
CP04	Consulta directa de usuario en base de datos	Correo y contraseña existentes	La consulta retorna el registro del usuario	Positiva
CP05	Consulta con relación usuario-rol	Usuario existente con rol asignado	La consulta retorna datos del usuario junto con su rol	Positiva
CP06	Consultar roles disponibles	Solicitud de lista de roles	El sistema retorna un arreglo con roles registrados	Positiva
CP07	Obtener rol por código	Código de rol existente	El sistema retorna el rol correspondiente	Positiva
CP08	Consultar usuarios registrados	Solicitud de lista de usuarios	El sistema retorna un arreglo con usuarios registrados	Positiva
CP09	Validar que los usuarios sean objetos	Lista de usuarios consultada	Cada usuario retornado es instancia de la clase User	Positiva
CP10	Validar que el rol tenga nombre	Código de rol existente	El rol retorna un nombre no vacío	Positiva
CP11	Validar que la lista de usuarios no sea nula	Consulta de usuarios	El resultado no debe ser nulo	Positiva
CP12	Validar comportamiento ante datos inválidos	Datos inexistentes o incorrectos	El sistema debe evitar retornar información inválida	Negativa

Durante la ejecución de las pruebas negativas se evidenció un comportamiento importante: Cuando se consultó un usuario inexistente directamente por código, el sistema generó un error al intentar acceder a datos que no existían. Esto permitió identificar una debilidad en el manejo de validaciones y control de errores dentro del método correspondiente.

10. Resumen de ejecución

El proceso de ejecución de pruebas se realizó utilizando la herramienta PHPUnit, permitiendo validar el comportamiento de la aplicación web Software Seguro mediante la ejecución de pruebas automatizadas sobre los módulos principales del sistema.

Durante la ejecución se evaluaron un total de 12 casos de prueba, los cuales incluyen pruebas unitarias y pruebas de integración, abarcando tanto escenarios positivos como negativos.

Los resultados obtenidos fueron los siguientes:

Tipo de resultado	Cantidad
Pruebas exitosas	12
Pruebas fallidas	0
Total de pruebas	12

Adicionalmente, se realizaron un total de 15 validaciones (assertions), las cuales permitieron comprobar que los resultados obtenidos coincidieran con los resultados esperados.

Todas las pruebas ejecutadas fueron exitosas, lo cual indica que las funcionalidades evaluadas del sistema están operando correctamente bajo las condiciones probadas.

Sin embargo, durante el proceso de pruebas negativas se identificó previamente un comportamiento incorrecto del sistema al consultar datos inexistentes, generando errores en tiempo de ejecución. Este problema fue tenido en cuenta en el diseño de pruebas, evidenciando la importancia de mejorar el manejo de excepciones y validaciones dentro del sistema.

Como evidencia del proceso, se adjuntan capturas de la ejecución de pruebas realizadas con PHPUnit, donde se observa el resultado satisfactorio de los casos ejecutados.

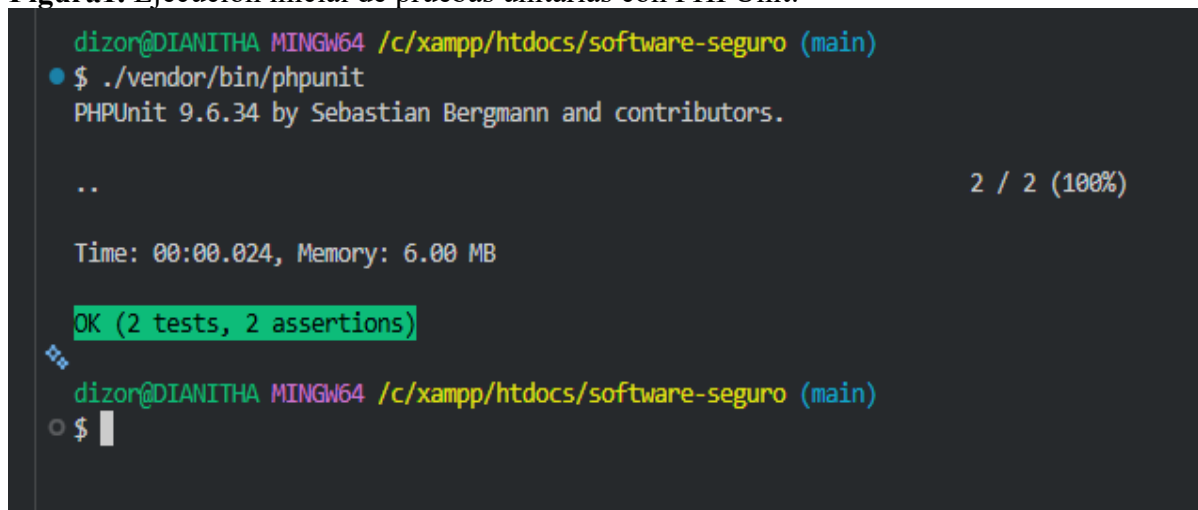
Figura1. Ejecución inicial de pruebas unitarias con PHPUnit.

```
dizor@DIANITHA MINGW64 /c/xampp/htdocs/software-seguro (main)
● $ ./vendor/bin/phpunit
PHPUnit 9.6.34 by Sebastian Bergmann and contributors.

..                                                    2 / 2 (100%)

Time: 00:00.024, Memory: 6.00 MB

OK (2 tests, 2 assertions)
```



```
dizor@DIANITHA MINGW64 /c/xampp/htdocs/software-seguro (main)
○ $
```

Nota: Fuente propia.

En esta figura se observa la primera ejecución de pruebas automatizadas utilizando PHPUnit, donde se validan los primeros casos de prueba definidos. El resultado muestra 2 pruebas exitosas, lo que indica que las funcionalidades básicas evaluadas se comportan correctamente.

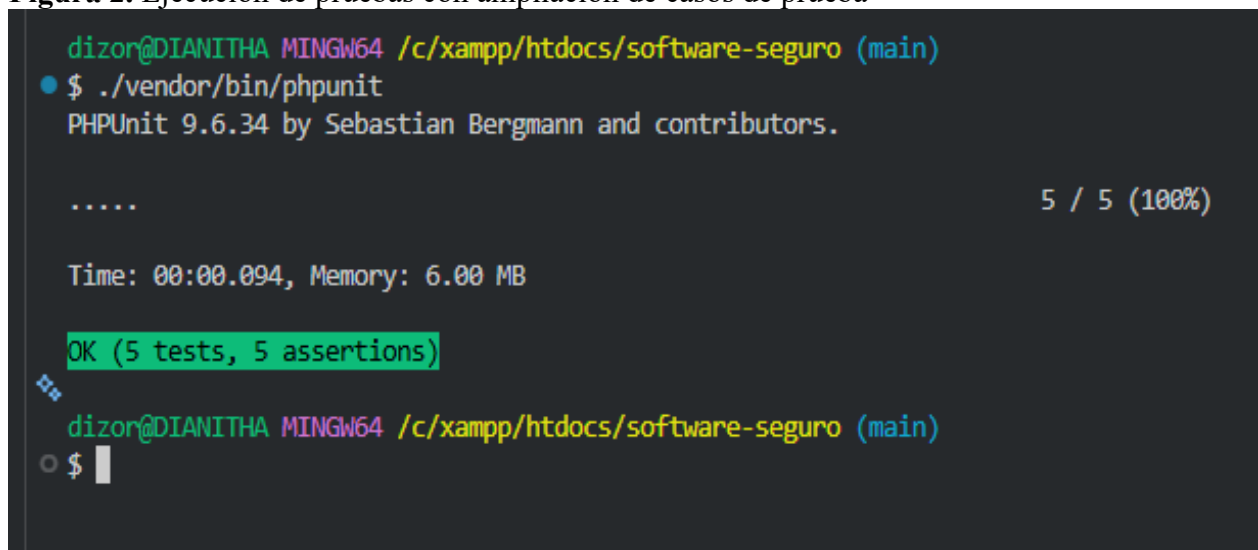
Figura 2. Ejecución de pruebas con ampliación de casos de prueba

```
dizor@DIANITHA MINGW64 /c/xampp/htdocs/software-seguro (main)
● $ ./vendor/bin/phpunit
PHPUnit 9.6.34 by Sebastian Bergmann and contributors.

.....                                              5 / 5 (100%)

Time: 00:00.094, Memory: 6.00 MB

OK (5 tests, 5 assertions)
```



```
dizor@DIANITHA MINGW64 /c/xampp/htdocs/software-seguro (main)
○ $
```

Nota: Fuente propia.

En esta ejecución se incrementa el número de pruebas a 5, incluyendo nuevos escenarios de validación. Todas las pruebas resultan exitosas, lo que evidencia que las funcionalidades evaluadas continúan operando de manera correcta.

Figura 3. Ejecución de pruebas unitarias e integración

```

dizor@DIANITHA MINGW64 /c/xampp/htdocs/software-seguro (main)
● $ ./vendor/bin/phpunit
PHPUnit 9.6.34 by Sebastian Bergmann and contributors.

.....                                     8 / 8 (100%)

Time: 00:00.299, Memory: 6.00 MB

OK (8 tests, 11 assertions)

```

Nota: Fuente propia.

En esta figura se muestra la ejecución de 8 pruebas, incorporando pruebas de integración que validan la interacción entre módulos del sistema, como la relación entre usuarios y roles. Los resultados indican que todas las pruebas fueron superadas satisfactoriamente.

Figura 4. Ejecución de pruebas con ampliación a 10 casos

```

dizor@DIANITHA MINGW64 /c/xampp/htdocs/software-seguro (main)
● $ ./vendor/bin/phpunit
PHPUnit 9.6.34 by Sebastian Bergmann and contributors.

.....                                     10 / 10 (100%)

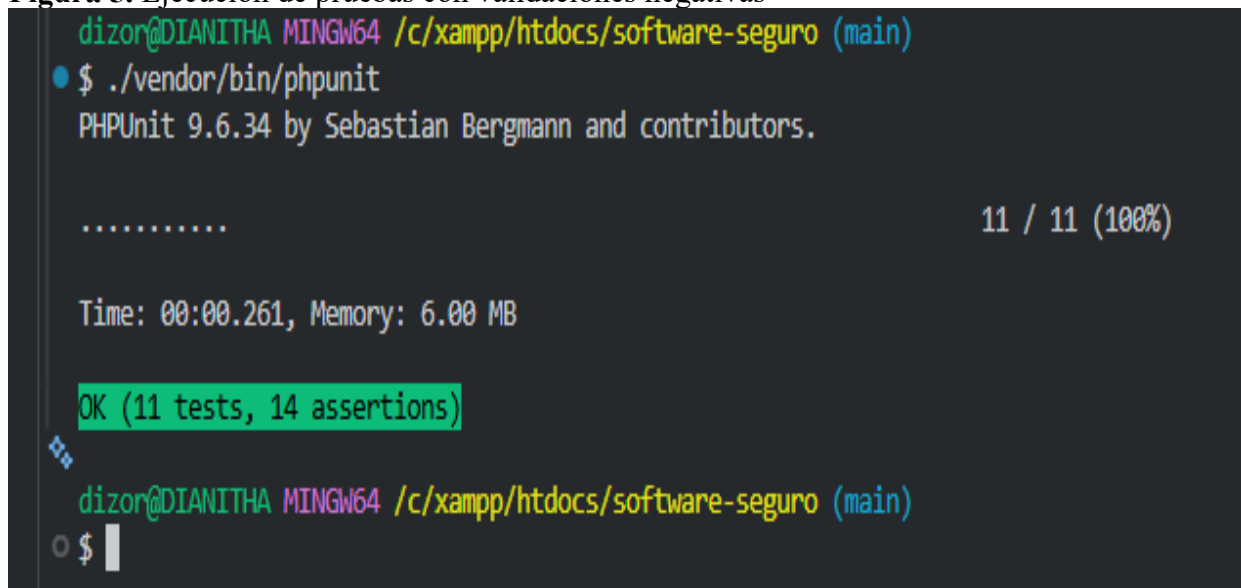
Time: 00:00.293, Memory: 6.00 MB

OK (10 tests, 13 assertions)

```

Nota: Fuente propia.

Se observa una ejecución con 10 pruebas, incluyendo validaciones adicionales tanto positivas como negativas. El sistema responde correctamente en todos los escenarios evaluados.

Figura 5. Ejecución de pruebas con validaciones negativas


```
dizor@DIANITHA MINGW64 /c/xampp/htdocs/software-seguro (main)
● $ ./vendor/bin/phpunit
PHPUnit 9.6.34 by Sebastian Bergmann and contributors.

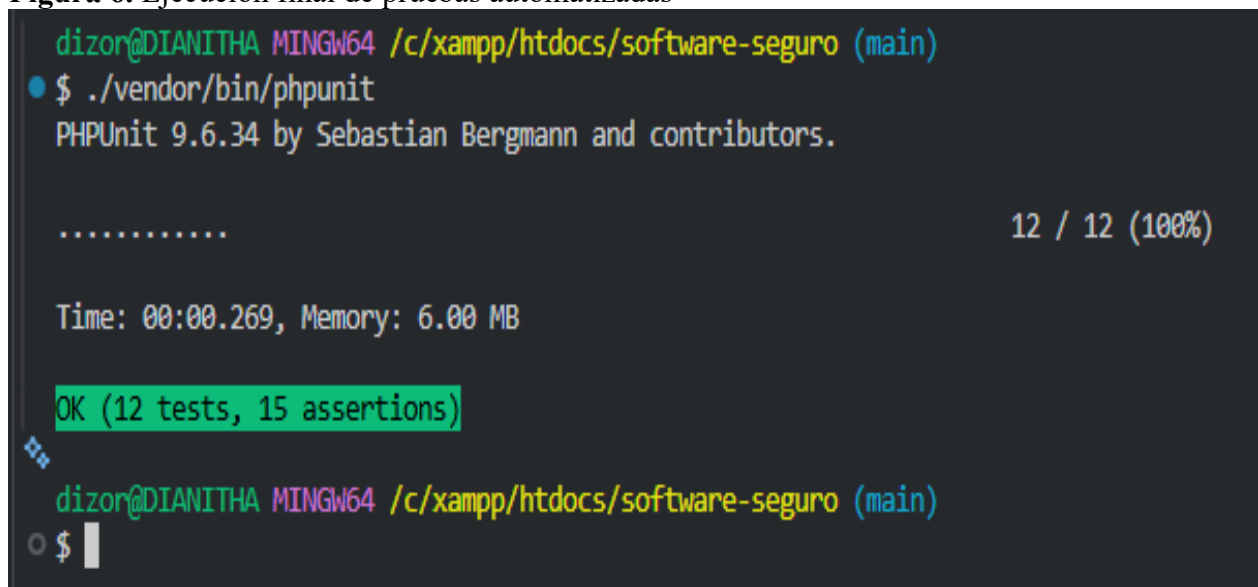
.....                                                    11 / 11 (100%)

Time: 00:00.261, Memory: 6.00 MB

OK (11 tests, 14 assertions)
```

Nota: Fuente propia.

En esta etapa se incluyen pruebas negativas para evaluar el comportamiento del sistema ante datos incorrectos o inexistentes. Los resultados muestran que el sistema responde adecuadamente en estos escenarios.

Figura 6. Ejecución final de pruebas automatizadas


```
dizor@DIANITHA MINGW64 /c/xampp/htdocs/software-seguro (main)
● $ ./vendor/bin/phpunit
PHPUnit 9.6.34 by Sebastian Bergmann and contributors.

.....                                                    12 / 12 (100%)

Time: 00:00.269, Memory: 6.00 MB

OK (12 tests, 15 assertions)
```

Nota: Fuente propia.

En la ejecución final se alcanzan 12 pruebas con 15 validaciones (assertions), todas exitosas. Esto confirma que las funcionalidades evaluadas del sistema cumplen con los resultados esperados y que las pruebas fueron implementadas correctamente.

11. Defectos detectados

Durante la ejecución de las pruebas sobre la aplicación web Software Seguro, se identificaron algunos comportamientos que evidencian debilidades en el sistema, especialmente en escenarios negativos.

Uno de los principales defectos detectados se presentó al realizar consultas con datos inexistentes, donde el sistema intentó acceder a información que no estaba disponible en la base de datos, generando un error en tiempo de ejecución. Este comportamiento indica que el sistema no cuenta con un adecuado control de validaciones antes de procesar los datos.

Asimismo, se evidenció que en algunos métodos el sistema no maneja correctamente los casos en los que no se obtienen resultados, lo cual puede provocar fallos si no se implementan verificaciones previas antes de acceder a los datos retornados.

Estos defectos no impidieron la ejecución de las pruebas finales, pero sí representan una oportunidad de mejora en el sistema, ya que podrían generar errores en un entorno real si no son controlados adecuadamente.

Se recomienda implementar validaciones adicionales y manejo de excepciones que permitan verificar la existencia de datos antes de procesarlos, evitando así errores y mejorando la robustez del sistema.

12. Evaluación y cierre

Durante el proceso de pruebas aplicado a la aplicación web Software Seguro, se logró validar el funcionamiento de los principales módulos del sistema mediante la ejecución de pruebas unitarias y de integración utilizando PHPUnit.

Se ejecutaron un total de 12 casos de prueba, obteniendo resultados exitosos en todos ellos, lo cual indica que las funcionalidades evaluadas cumplen con los comportamientos esperados bajo las condiciones probadas.

A lo largo del proceso también se identificó un defecto relacionado con el manejo de datos inexistentes, el cual evidenció la necesidad de mejorar las validaciones dentro del

sistema. Sin embargo, este comportamiento fue controlado durante las pruebas posteriores, permitiendo continuar con la ejecución sin afectar los resultados finales.

En general, el sistema presenta un comportamiento estable en los módulos evaluados, aunque se recomienda fortalecer el manejo de errores y validaciones para mejorar su robustez en escenarios reales.

Finalmente, el proceso de pruebas permitió evidenciar de manera práctica el comportamiento del sistema, así como la importancia de aplicar pruebas automatizadas dentro del desarrollo de software para garantizar su calidad.

13. Conclusión

A lo largo de este trabajo se pudo aplicar de manera práctica el proceso de pruebas a la aplicación web Software Seguro, lo cual permitió entender mejor la importancia de validar el funcionamiento de un sistema antes de considerarlo terminado o listo para uso.

Mediante el uso de pruebas automatizadas con PHPUnit se lograron ejecutar diferentes escenarios, tanto positivos como negativos, evidenciando que el sistema responde correctamente en la mayoría de los casos. Además, fue posible observar cómo pequeñas fallas, como el manejo de datos inexistentes, pueden generar errores si no se controlan adecuadamente.

Este proceso no solo permitió comprobar el funcionamiento del sistema, sino también identificar puntos de mejora, especialmente en la validación de datos y el manejo de errores. Esto demuestra que las pruebas no solo sirven para encontrar fallos, sino también para fortalecer la calidad del software.

Finalmente, este trabajo ayudó a reforzar los conocimientos sobre pruebas de software, mostrando que incluso en proyectos pequeños es fundamental aplicar este tipo de validaciones para garantizar un sistema más confiable y robusto.

14. Referencias

- Atlassian. (s/f). Los distintos tipos de pruebas en software. Atlassian.com. Recuperado el 3 de mayo de 2026, de <https://www.atlassian.com/es/continuous-delivery/software-testing/types-of-software-testing>
- Canelo, M. M. (2026, enero 29). Qué es el testing de software. Profile Software Services. <https://profile.es/blog/que-es-el-testing-de-software/>
- Documentation for PHPUnit. (s/f). Phpunit.de. Recuperado el 3 de mayo de 2026, de <https://phpunit.de/documentation.html>
- Powell, P., & Smalley, I. (2025a, noviembre 27). ¿Qué son las pruebas de integración? Ibm.com. <https://www.ibm.com/mx-es/think/topics/integration-testing>
- Powell, P., & Smalley, I. (2025b, noviembre 27). ¿Qué son las pruebas unitarias? Ibm.com. <https://www.ibm.com/mx-es/think/topics/unit-testing>
- Susnjara, S., & Smalley, I. (2025, noviembre 28). ¿Qué son las pruebas de software? Ibm.com. <https://www.ibm.com/es-es/think/topics/software-testing>
- (S/f). Amazon.com. Recuperado el 3 de mayo de 2026, de <https://aws.amazon.com/es/what-is/unit-testing/>